

VANGUARD

One picture. Every source. Zero delay.

Master Guidelines & Learning Guide

Multi-Source Defence Situational Awareness System
Problem ID D-05 · HackHertz 2026 Defense Track
Team Destroyer of Worlds · Document version 1.1

How to read this document

This is the single document to hand a new team member, and the one to revise from before judging. It teaches the **concepts** behind VANGUARD, not just its API surface.

Part	Covers	Read if you are...
I	Domain foundations — what a COP is, why fusion is hard	New to defense systems
II	The engineering principles	Extending the codebase
III	The mathematics, taught from first principles	Explaining confidence to a judge
IV	Failure modes we hit and fixed	About to change fusion constants
V	Operating and demonstrating the system	Presenting
VI	Judge Q&A preparation	Presenting

PART I — DOMAIN FOUNDATIONS

1.1 What is a Common Operating Picture?

A **Common Operating Picture (COP)** is a single, shared, continuously-updated representation of an operational situation that every decision-maker sees identically.

The word doing the work is **common**. When a radar operator, a duty officer and a commander each read a different screen, they form different mental models, and coordination degrades exactly when it matters most. A COP exists so that when the commander says "the contact in Sector 4", everyone is looking at the same contact.

The failure a COP prevents: in a real operations centre, information arrives asynchronously from systems that were procured separately, speak different formats, and disagree. A watchstander manually cross-references a radar screen, a weather terminal, a personnel tracker, a log console and a radio net. Each context switch costs seconds and attention. During a fast-developing situation those seconds are the whole problem.

1.2 Why multi-source fusion is genuinely hard

It is tempting to think fusion means "put all the data on one screen". That is **aggregation**, and it makes the overload worse — now one screen has five times the noise.

Fusion means answering: *which of these observations are about the same thing?*

Four hard problems sit underneath:

1. The correspondence problem. A radar return at 23.02N, 72.57E at 11:42:03 and a perimeter sensor trip at 23.03N, 72.58E at 11:42:31. One event or two? There is no ID field linking them. You must infer correspondence from space, time and physics.

2. Conflicting evidence. Radar holds a contact; the patrol at that location reports nothing. Which is right? Neither, necessarily — the contact may be behind terrain. A fusion system must represent *degrees of belief*, not

booleans.

3. Variable reliability. A calibrated radar and an anonymous phone tip are not equal evidence. Nor is a five-second-old reading equal to a forty-minute-old one. Fusion must weight sources and decay observations.

4. False corroboration. The most dangerous failure. If the same radar reports the same contact six times and the system counts six independent confirmations, it will report high confidence in something one instrument saw once. **Redundancy masquerading as independence is worse than no fusion at all**, because it produces confident wrong answers.

VANGUARD hit exactly this during development. See Part IV, Failure 2.

1.3 Why generic AI tools fail at this

An LLM handed raw sensor feeds will produce a fluent, plausible, unverifiable summary. Three specific failure modes:

Failure	What it looks like	Why it disqualifies the tool
Fabricated specifics	"Three contacts approaching from the north-east at 240 knots" when there were two at 180	An officer may act on it
Unfalsifiable confidence	"High confidence of a perimeter breach"	No way to check where the number came from
Invented citations	References event IDs that do not exist	Looks <i>more</i> rigorous while being less

A defense operator cannot verify a claim they cannot trace. **An unverifiable summary is worse than no summary**, because it carries the authority of a system without the accountability of one.

VANGUARD's answer: the model never produces a number, and every sentence it writes is mechanically checked against the event store before an operator sees it.

PART II — ENGINEERING PRINCIPLES

2.1 The central thesis

The fusion engine is the product. The language model is a presentation layer.

Test it: delete the API key. VANGUARD produces a complete briefing with real citations, ranked courses of action with honest tradeoffs, and correct confidence math. `provenance.engine` reads `deterministic` and nothing else changes.

That is the difference between a system and a prompt.

2.2 The seven principles

P1 — Determinism before intelligence

Anything computable deterministically is computed deterministically. Confidence, severity, anomaly flags and threat posture are arithmetic. Only phrasing is delegated.

P2 — Explainability is a feature, not documentation

Every score decomposes into named factors, served at GET /api/v1/events/:id/correlations alongside a **counterfactual**: what the score would be without corroboration. That single number answers "what did fusion actually buy me here?"

P3 — Graceful degradation everywhere

Every dependency has a fallback, and the fallback is honest about itself:

Dependency	Fallback chain
Gemini	retry ×2 → deterministic synthesizer
Open-Meteo	cache → synthetic climatology
A feed	health degrades → reliability drops → confidence falls

P4 — Independence over volume

Corroboration from a different source counts fully; from the same source, 0.35. Ten radar returns are one instrument's opinion. Radar plus a thermal trip is evidence.

P5 — Reproducibility

Every simulator draws from a seeded generator. Same SIM_SEED, same scenario, every machine. A demo you cannot rehearse is a demo you cannot trust.

P6 — Zero-friction operation

npm install && npm run dev. No key, no database, no Docker. A judge with four minutes and a laptop must be able to run it.

P7 — Auditability

Every escalation records its trigger events. Every briefing records its engine, latency, and **how many invented citations were stripped**. Nothing changes state without leaving a trace.

2.3 Layer discipline

Layer	Owns	Must never
Ingestion	Polling, liveness, raw payloads	Build a UnifiedEvent
Normalization	Shape conversion, base severity	Assign confidence
Fusion	Correlate, score, flag, escalate	Touch the network
AI	Language and phrasing	Decide a number
API/WS	Serialization, filtering	Compute anything

The rule that keeps this honest: only fusion/confidence.ts may produce a confidence number; only fusion/severity.ts may change a severity.

PART III — THE MATHEMATICS

Full derivations in FUSION_MATH.md. This part teaches the *reasoning*.

3.1 Confidence, from first principles

The question: given an observation, how much should a commander believe it?

Three things determine that, and they **multiply** rather than add — because each is a necessary condition. A perfectly reliable sensor reporting something an hour ago should not score highly, and no amount of corroboration rescues a reading nobody trusts. Addition would let a strong factor rescue a fatally weak one.

$$\text{Confidence} = \min(100, \text{round}(R_s \times D_t \times B_c \times 100))$$

Factor 1 — Source reliability R_s

How much do we trust this instrument at all?

Feed	Weight	Why
Weather	0.95	Real measured API, calibrated
Radar	0.92	Calibrated, but clutter and ghosts
Personnel	0.88	Accurate but sparse, latency-prone
Log	0.80	Deterministic, but false trips
Incident	0.72	Human-reported: highest variance

Learning point: the ordering is a **judgement written down**. Instrumented sensors with calibrated error models outrank machine logs, which outrank unverified human reports. Writing it down is what makes it arguable — and therefore defensible.

Factor 2 — Recency decay D_t

Information ages. Exponential decay with a 15-minute half-life:

$$D_t = e^{-\lambda \Delta t}, \quad \lambda = \ln 2 / 900$$

Why exponential rather than linear? Because relevance does not fall off a cliff at an arbitrary cutoff, and it should never reach exactly zero — a stale observation is weaker, not meaningless.

Why 15 minutes? It matches watch-floor tempo: older than that and you re-verify before acting.

Elegant side effect: the threat score decays on its own as a situation goes quiet. No separate expiry job.

Factor 3 — Corroboration boost B_c

$$B_c = \min(1.6, 1 + 0.15(N_{\text{eff}} - 1))$$

The cap is the important part. Without it, twenty correlated low-grade reports could manufacture certainty out of nothing — the false-corroboration failure from §1.2. The 1.6 ceiling means five independent sources saturate the term and a sixth adds nothing.

N_{eff} discounts same-source confirmation to 0.35, encoding P4.

The counterfactual

The most persuasive number VANGUARD shows is a subtraction:

confidence with corroboration:	100%
confidence without:	92%
gain:	+8

It converts an abstract claim ("we fuse sources") into a measured quantity.

3.2 Correlation: two windows and a closure

Two events correlate when **both** hold: within 5 km **and** within 600 s.

Why both? Either alone is meaningless. Everything in an AO is within 45 km of everything else eventually; everything within an hour is within an hour of something.

Why transitive closure? A and C are 8 km apart — no direct link. But B sits between and links to both. All three belong to one developing picture. That is precisely the chain of reasoning a human analyst performs, so the engine performs it too.

But transitivity is dangerous — see Part IV, Failure 3.

Complexity: a spatial grid with cell size = correlation radius reduces the naive $O(n^2)$ scan to roughly $O(n)$. Measured: 803 comparisons for 118 events instead of ~7,000.

3.3 Anomaly detection: why statistics, not the model

$z = \frac{x - \mu}{\sigma}$, $\text{flag when } |z| \geq 2.5$

Three detectors: **rate** (is this feed unusually chatty against its own history?), **kinematic** (is this contact's speed an outlier?), **spatial** (is this event unusually isolated?).

Two subtleties worth learning:

Exclude the sample from its own baseline. Otherwise a single extreme value drags the mean toward itself and hides.

Use a robust z-score as well. The median/MAD variant, scaled by 0.6745 to match the classical score for normal data, is not fooled when one prior spike has already inflated σ — the classic way an outlier detector misses the second outlier.

3.4 Threat posture: why superlinear weights

$\text{score} = \sum W(\text{severity}) \times \frac{\text{confidence}}{100} \times D_t$

with weights low 0.25, medium 1.5, high 7, critical 15.

Why so steep? Because posture must track **threats, not volume**. A watch floor tracking 200 routine contacts is not at higher alert than one tracking 20. With flatter weights, routine traffic accumulates a permanent floor and GREEN becomes unreachable — see Part IV, Failure 6.

Hysteresis. Escalation is immediate; de-escalation requires falling 15% below the threshold. Without it the indicator flickers at a boundary, which both looks broken and trains operators to ignore it. *An alarm that cries wolf is worse than no alarm.*

PART IV — FAILURE MODES WE HIT AND FIXED

The most valuable part of this document. Every one of these was found by running the system, not by reading the code. Each has a regression test.

Failure 1 — Escalation compounded across ticks

Symptom: within a minute, 194 of 285 events read CRITICAL. Threat score 103 at boot.

Cause: the pipeline re-fuses the whole picture every tick over the same objects. Escalation raised severity from the *current* value, so medium → high → critical in three ticks **with no new evidence**.

Fix: reset severity = baseSeverity at the start of the stage. Escalation is now idempotent — re-running converges instead of ratcheting.

Lesson: *any operation applied repeatedly to persistent state must be idempotent*. Ask of every mutation: what happens if this runs a thousand times?

Failure 2 — Entity duplication caused false corroboration

Symptom: 188 radar events for 10 actual contacts within 50 seconds.

Cause: radar re-emitted every track each sweep as a **new event**. Those near-identical re-reports then correlated *with each other*.

This is §1.2's false-corroboration problem appearing in our own system — the exact failure the engine exists to prevent, produced by an ID-generation choice.

Fix: entity-state observations derive a **stable ID** from the entity key and update one event in place. A COP holding ten contacts shows ten contacts.

Lesson: *distinguish "a thing that exists" from "a thing that happened"*. Conflating them is the most damaging modelling error available in a fusion system.

Failure 3 — Cluster-wide escalation over-escalated the board

Symptom: 37 of 45 events escalated by fusion.

Cause: the rule escalated every member of a cluster with ≥ 3 distinct sources. But clusters are **transitive closures** — a chain of overlapping 5 km links spans tens of kilometres. A distant contact linked through two intermediaries inherited "confirmed by three sources" without anything having confirmed it.

Fix: test the event's **own** corroboration links.

Lesson: *a property of a group is not automatically a property of its members*. Transitive grouping is useful for display and dangerous for inference.

Failure 4 — Uncorroborated contacts promoted to CRITICAL

Symptom: Uncorrelated vessel contact R-1003, conf=92, corr=0 → CRITICAL.

Cause: promotion required only severity=high and confidence ≥ 85 . But a fresh reading from a reliable sensor scores ~92% on reliability and recency **alone**. Every uncorroborated radar return became CRITICAL on arrival.

Fix: require at least one **independent** corroborator.

Lesson: *confidence is not evidence of reality*. A high score can mean "this sensor is trustworthy and the reading is fresh" — not "this is definitely happening". One instrument's unconfirmed opinion is not a command emergency, however good the instrument.

Failure 5 — Escalation rules chained within one pass

Symptom: NET-1101 Backhaul link latency — a routine network log — reached CRITICAL.

Cause: Rule 1 promoted it medium → high; Rule 2 then saw high and promoted it again. Two tiers in one pass from a latency warning.

Fix: gate Rule 2 on baseSeverity === 'high' — what the **source** claimed, not the post-Rule-1 value.

Lesson: *when rules compose, define what each reads explicitly.* Sequential rules over shared mutable state chain in ways neither author intended.

Failure 6 — Posture pinned at RED with nowhere to escalate

Symptom: boots straight to RED and stays there.

Cause: thresholds calibrated for a much smaller picture; routine low/medium traffic accumulated a permanent floor of ~45 points.

Fix: superlinear severity weights, thresholds recalibrated against the **measured** steady state, and a more realistic 12% non-cooperative contact rate in the simulator.

Lesson: *calibrate against measured behaviour, not intuition.* And a system that starts at maximum alert has destroyed its own signal.

Failure 7 — A feed silently empty at startup

Symptom: the incident feed showed nothing for the first ~15 seconds.

Cause: routine reporting is stochastic and sparse by design.

Fix: seed a small backlog in `init()`, as radar already did for tracks.

Lesson: *a demo's first ten seconds are its most scrutinised.* An empty panel reads as a broken integration regardless of the cause.

PART V — OPERATING THE SYSTEM

5.1 Running it

```
cd server
npm install
npm run dev          # http://localhost:3001/api/v1 · ws://localhost:3001/stream
```

No key required. With one:

```
cp .env.example .env  # add GEMINI_API_KEY
```

5.2 Pre-demo checklist

```
npm run typecheck  # clean
npm test           # 114 passing
npm run smoke      # all invariants hold
```

Then confirm the live baseline:

```
curl localhost:3001/api/v1/events/stats
```

Healthy: ~45 events, mostly low/medium, a few high, 0–3 critical, posture GREEN or YELLOW. **Unhealthy:** everything CRITICAL → a compounding bug is back (Failure 1 or 3).

5.3 The demo sequence

#	Action	What to say
1	Open the command center	"Five feeds, one picture. Weather is genuinely live from Open-Meteo."
2	Click a corroborated contact	"Three independent feeds agree. Radar, perimeter sensor, patrol sighting."
3	Open the Explainability Drawer	"Reliability × recency × corroboration. Without corroboration this would be 92%. Fusion bought us 8 points. "
4	Omnibar query	"Plain English to structured filter — in two milliseconds, no API call."
5	Read the briefing	"Every claim cites event IDs. Watch —" click a citation, it opens the raw event.
6	POST /ai/verify	"verified: true, zero ungrounded citations. Independently checkable."
7	Trigger border_spike	"Seven correlated reports plus two fast contacts. Watch the posture." → ORANGE → RED
8	Engage degraded comms	"Confidence across the whole picture falls 93 to 43. Not a banner — the math."
9	Delete the API key, re-brief	"Still a full briefing with real citations. The fusion engine is the product."

Step 9 is the strongest moment available. Do not skip it.

5.4 If something goes wrong live

Problem	Response
Weather layer empty	It cannot be — cache then synthetic. Say "that's the fallback working."
Gemini slow	It is cached and pre-generated; the panel never blocks.
Posture stuck GREEN	Trigger a scenario. That is what it is for.
Something genuinely breaks	POST /api/v1/simulation/reset

PART VI — JUDGE Q&A PREPARATION

"Is this just a wrapper around Gemini?" No — and I can show you. *(Remove the key, regenerate.)* Full briefing, real citations, provenance.engine: "deterministic". The fusion engine is 100% deterministic code with 114 tests.

"How do you know the AI isn't hallucinating?" We assume it is. Every citation is resolved against the store; invented IDs are stripped; claims left uncited are discarded; the counts are displayed. POST /api/v1/ai/verify re-checks it independently.

"Where does 88% come from?" reliability × recency × corroboration. *(Open the drawer.)* All three factors, plus a counterfactual showing what corroboration contributed. GET /api/v1/intelligence/config

serves the live constants.

"Is the data real?" Weather is genuinely live. The other four are high-fidelity simulators — persistent kinematic tracks, modelled sensor false-alarm rates, reporter credibility — because real feeds are classified. Every adapter implements one interface; swapping in a real feed is one file.

"What happens at 10,000 events?" A fusion pass over 118 events is 2–37 ms against a 3000 ms tick budget — roughly 50× headroom. Correlation is grid-indexed, so it is near-linear rather than quadratic. Beyond that, the store becomes the constraint and would move to a spatial database.

"What if two sources conflict?" Conflict is represented, not resolved. Neither is discarded; both appear, and the uncorroborated one simply scores lower. The briefing is instructed to state uncertainty explicitly. Hiding a disagreement is how you lose an operator's trust.

"Could this be used offensively?" No, and it is constrained in code. The COA generator is restricted by system instruction to observation, verification, reinforcement, evacuation and deconfliction. It has no targeting function and no weapons interface.

"What would you do with another month?" Persist the event store for cross-session replay; add a real track-history buffer so the time-scrubber replays contact trajectories, not just discrete events; and formalise the affinity matrix by learning it from labelled corroboration outcomes instead of hand-setting it.

Appendix — Constants quick reference

Constant	Value	Where
Correlation radius ΔR	5000 m	CORRELATION_RADIUS_METERS
Correlation window ΔT	600 s	CORRELATION_WINDOW_SECONDS
Recency half-life	900 s	REGENCY_HALF_LIFE_SECONDS
Corroboration boost	+0.15/source, cap 1.6	CORROBORATION_BOOST_PER_SOURCE
Same-source weight	0.35	SAME_SOURCE_CORROBORATION_WEIGHT
Dedupe window	150 m / 30 s	DEDUPE_*
Anomaly threshold	\	z\
Escalation sources	3 distinct	ESCALATION_DISTINCT_SOURCES
Critical promotion	$\geq 85\%$ + corroboration	CRITICAL_PROMOTION_CONFIDENCE
Severity weights	0.25 / 1.5 / 7 / 15	SEVERITY_WEIGHT
Threat thresholds	25 / 60 / 110	THREAT_THRESHOLDS
Hysteresis	15%	THREAT_HYSTERESIS
Store capacity	5000 events	EVENT_STORE_CAPACITY
Active horizon	3600 s	EVENT_ACTIVE_HORIZON_SECONDS

All served live at `GET /api/v1/intelligence/config`.